

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

05/09/2023

Last (family) Name _____

First (given) Name _____

Matricola

Codice Persona

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Did you participate in the extra points practical test? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading [] Yes, Motivation:

[] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 18 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

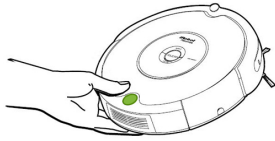
- **DISTANCE MODE ONLY**

- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone open, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)



Consider floor cleaning robots, automated devices designed to maintain and clean floors without human intervention. Utilizing a combination of sensors, algorithms, navigation systems, and video cameras, these robots can map out rooms and detect dirt, debris, and obstacles, ensuring efficient cleaning. They can be set to clean at specific times or when the owner is away. Their sleek and compact design allows them to access hard-to-reach areas, such as underneath furniture, ensuring a comprehensive cleaning experience.

The robots must be connected to a wireless network and linked to the robot's company cloud where the robot stores all the data needed for cleaning activities (e.g., mapped rooms, scheduled activities). Many advanced models can be controlled via smartphone apps, which communicate with the company cloud infrastructure, allowing users to set cleaning schedules, monitor progress -- also through live video streaming--, or manually direct the robot from their phones.

[2 point] 1. Suggest two relevant assets at risk in such a scenario. Motivate your answer.

Asset	Motivation
Reputation	[...]
User private data: "habits", mapped rooms, "away" times, videos from the house	[...]

[2 points] 2. Suggest the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	Threat Agents
Denial of service Motivation: [...]	Competitor Motivation: [...]
Data breach Motivation: [...]	Cyberattackers Motivation: [...]

Exercise 2 - Introduction to Cryptography (6 points)

Consider the case of a public software archive: the software distributor has designed the following highly efficient scheme to protect the integrity of the distributed software. The distributor claims that the scheme avoids the issues in key management, is fast, and guarantees both the integrity of the distributed software and its origin.

The scheme goes as follows: given a piece of software (e.g. a Linux distribution package), consider it as a (long) bitstring s and distribute the bitstring obtained concatenating the strings a, s, b, c , where:

- a is obtained as $\text{SHA-256}(s)$
- b is obtained as $\text{SHA-256}(s||a)$
- c is obtained as $\text{SHA-256}(a||s||b)$

[4 point] Argue on the fact that the proposed scheme provides integrity and origin authentication of the distributed software, either showing an attack against each property, or proving (an informal sketch is enough), that the scheme provides such property.

[Two points for each property] The scheme does not provide either of the claimed properties. Indeed, an attacker can modify the software bitstring s and recompute the correct values for the tampered software without any issues: it just runs the same computations as the distributor, voiding the integrity claim.

There is no mechanism in place to provide origin authentication; indeed an attacker can just take any piece of software of her/his liking and produce a distributable bundle compliant with the scheme.

[2 point] Design a simple and effective scheme to protect the integrity and authenticity of a piece of software which you are willing to distribute. Assume that the machines on which you are distributing your software have a network connection, and a reliable time reference.

A simple and effective scheme mimics the one in common Linux distribution: the software packages are signed by the person/entity building them with their private key, and the signature is checked when the software is obtained. The public keys with which the signature should be checked must be supplied in a trustworthy fashion, e.g., with the OS installation bundle.

Exercise 3 - Binary Vulnerabilities (6 points)

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04
05 typedef struct {
06     int isAdmin;
07     char feedback[32];
08     char username[32];
09 } UserProfile;
10
11 void processData(char *name, char *comment) {
12     UserProfile profile;
13     char feedbackMessage[128];
14
15     profile.isAdmin = 0; // By default, user is not an admin
16
17     strcpy(profile.username, name);
18     strcpy(profile.feedback, comment);
19
20     strcpy(feedbackMessage, "Feedback:");
21     strcat(feedbackMessage, profile.feedback);
22
23     printf(feedbackMessage);
24
25     if(profile.isAdmin) {
26         printf("You have administrative privileges!\n");
27         system("/bin/sh"); // Open a shell
28     } else {
29         printf("You are a regular user.\n");
30     }
31 }
32
33 int main() {
34     char name[48];
35     char comment[48];
36     clearenv(); // Clear all environment variables
37
38     printf("Enter your name: ");
39     fgets(name, sizeof(name), stdin);
40
41     printf("Any comments? ");
42     fgets(comment, sizeof(comment), stdin);
43
44     // [...] Code for removing newline characters (not shown)
45
46     processData(name, comment);
47     return 0;
48 }
```

Assume that:

- Environment variables are located in the highest addresses.
- *The program is compiled for the x86 architecture (32 bit) and for an environment that adopts the usual cdecl calling convention. Furthermore, assume that no compiler-level or OS-level mitigations against exploiting memory corruption errors are present (unless specified otherwise).*

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivate and Modify the line to solve the detected vulnerability
Buffer Overflow	[0.25] Lines 17, 18;	See Slides [0.25]
Format String	[0.25] Line 23;	See Slides [0.25]

From now on, assume that the “Stack Canary” mitigation technique is correctly implemented (i.e, it can not be guessed or copied).

[3 points] 2. An attacker wants to execute the following shellcode, composed of 8 bytes, which opens a shell: `0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`.

2.a) Which of the identified vulnerabilities can be exploited to execute the shellcode presented above? Motivate your answer.

[0.25] Vulnerability: Format string [see slides]

[0.75] Motivation: Buffer overflow cannot be exploited since the canary is enabled. By overflowing the buffer, we would overwrite the canary too. [see slides]

2.b) Describe **all the steps and assumptions** required to exploit the selected vulnerability successfully. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

- [0.25] Insert the shellcode in one of the buffers.
- [0.25] Insert the format string exploit in the comment variable which will be copied into the feedback variable and then in the feedbackMessage buffers.
- [0.25] For the exploitation, you must consider the byte already printed “Feedback:” for the ‘pos’ and address computation.
- [0.25] The format string will overwrite the sEIP of processData() with the address of the buffer in which you have placed the sheelcode.

2.c) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

		Before vuln line 23	CONTENT		After vuln line 23	
L						
^						
	ESP	feedbackmessage[0-3]	f e e d	processData()	f e e d	
		[...]	b a c k	processData()	b a c k	
		[...]	: FS exploit	processData()	: FS exploit	
		[...]	FS exploit	processData()	FS exploit	
		feedbackmessage[28- 31]		processData()		
		isAdmin	0	processData()	0	
		feedback[0-3]	FS exploit	processData()	FS exploit	
		[...]	FS exploit	processData()	FS exploit	
		feedback[28- 31]	FS exploit	processData()	FS exploit	
		username[0-3]	shellcode	processData()	shellcode	X
		[...]	shellcode	processData()	shellcode	
		username[28- 31]	shellcode	processData()	shellcode	
	EBP	sEBP		processData()		
		sEIP		main()	X	
		*name		main()		
		*comment		main()		
		comment[0-3]	FS exploit	main()	FS exploit	
		[...]	FS exploit	main()	FS exploit	
		comment[43- 47]	FS exploit	main()	FS exploit	
		name[0-3]	shellcode	main()	shellcode	
		[...]	shellcode	main()	shellcode	
		name[43- 47]	shellcode	main()	shellcode	
		sEBP		main()		
		sEIP				
		argc				
H		argv				

[2 points] 4. A friend of yours suggests that there is an alternative way to open a privileged shell, **without executing any shellcode or calling any library functions**. Please specify the exploited vulnerability and **all the steps and assumptions** required for successful exploitation. If it is not possible, please explain why.

- [0.5] Format string
- [0.5] Insert the format string exploit in the comment variable which will be copied into the feedback variable and then in the feedbackMessage buffers.
- [0.5] For the exploitation, you must consider the byte already printed “Feedback:” for the ‘pos’ and address computation.
- [0.5] The format string will overwrite isAdmin variable with 1.

Exercise 4 - Web Vulnerabilities (6 points)

PayDude is a secure and easy-to-use online payment platform that allows individuals and businesses to send and receive money instantly. Whether you're shopping online, freelancing, or running a business, PayDude offers a range of services like one-click payments, invoicing, and currency conversion to make financial transactions seamless.

The relevant code of the application is the following:

```
00 @app.route('/send_money', methods = ['POST'])
01 def send_money(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("/login"))
04     if "CSRF" not in request.keys():
05         return abort(403)
06     if request["CSRF"] != session["CSRF"]:
07         return abort(403)
08     if "receiver_id" not in request.keys():
09         return render_template("error.html", error="No receiver specified!")
10     if "amount" not in request.keys():
11         return render_template("error.html", error="No amount specified!")
12     sender = session["user_id"]
13     receiver = request["receiver_id"]
14     amount = request["amount"]
15     # We first check if there are enough money
16     check_s_query = "SELECT balance FROM USERS WHERE user_id = " + sender +
17         " AND balance < " + amount + ";"
18     result = db_execute_query(check_query)
19     # If one row is returned, there are not enough money
20     if len(result) == 1:
21         current_balance = xss_sanitizate(result[0]["balance"])
22         return render_template("home.html", error="Not enough money!" +
23             " Your balance is " + current_balance)
24     # Now we can update the sender (we also check amount > 0)
25     update_query = "UPDATE USERS SET balance = balance - ? WHERE user_id = ? AND ? > 0;"
26     db_execute_query_with_statement(update_query, amount, sender, amount)
27     # And finally the receiver (we also check amount > 0)
28     update_query = "UPDATE USERS SET balance = balance + ? WHERE user_id = ? AND ? > 0;"
29     db_execute_query_with_statement(update_query, amount, receiver, amount)
30     return render_template("home.html", success="Money transferred!")
31
32 @app.route('/friends', methods = ['GET'])
33 def friends(request, session):
34     if not is_session_valid(session):
35         return redirect(url_for("/login"))
36     user_id = session["user_id"]
37     # Retrieving friends info (user id and username)
38     friends_query = "SELECT friend_id FROM FRIENDS WHERE user_id = ?;"
39     friends = db_execute_query_with_statement(friends_query, user_id)
40     friends_info = []
41     for friend in friends:
42         friend_id = friend["friend_id"]
43         username_query = "SELECT username FROM USERS WHERE user_id = ?;"
44         results = db_execute_query_with_statement(username_query, user_id)
45         friends_info += [friend_id, results[0]["username"]]
46     # Setting CSRF token
47     session["CSRF"] = generate_token(2)
48     return render_template("friends.html", CSRF=session["CSRF"], friends=friends_info)
```

```
47
48 def generate_token(lenght):
49     # This function generates a random token of the provided length
50     # We only use digits, e.g., 0123456789
51     token = ''.join(random.choice(string.digits) for i in range(length))
52     return token
...
70 # friends.html
71 <html>
...
80     {% for friend in friends %} # Iterating over friends
81         <h7>{{ xss_sanitize(friend[1]) }}</h7> # Displaying username
82         <form method="POST" action="/send_money">
83             <p>Amount:</p><input type="text" name="amount">
84             <input type="hidden" name="receiver_id" value="{{ friend[0] }}">
85             <input type="hidden" name="CSRF" value="{{ CSRF }}">
86             <input type="submit" value="Send money">
87         </form>
88     {% endfor %}
...
100 </html>
```

The relational database schema used by the website is the following:

Table: **USERS**

user_id (Primary Key) (Integer) (Auto increment)	username (String)	hashed_password (String)	balance (Integer)
--	----------------------	-----------------------------	----------------------

Table: **FRIENDS**

user_id (String) (Primary Key) (Referencing USERS.user_id)	friend_id (String) (Primary Key) (Referencing USERS.user_id)
--	--

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **render_template** **does not perform any sanitization of the input**.
- The **DBMS** does not support stacked queries (e.g., "SELECT ... ; INSERT INTO ... ;" fails if executed)
- The password is hashed using best practices, i.e., it is salted and hashed using SHA256. Recovering the user's password knowing only the hash would typically require a computationally expensive brute-force attack.
- The function **xss_sanitize** correctly escapes all the html entities, i.e., all the html special characters are converted to their equivalent data (> becomes >).

More details:

- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
SQL Injection (SQL-I)	<i>Line 16. An attacker can read data from the DB such as hashed passwords.</i>	<i>Using prepared statement.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>Not present.</i>	<i>Nothing to do.</i>
Cross-site request forgery (CSRF)	<i>The CSRF is random but it not secure since is composed by only 2 digits. Hence, a CSRF attack will work once every 100 clicks on a malicious page containing a CSRF attack.</i>	<i>Increase the number of characters for a CSRF token by increasing the length of the token and the available characters, i.e., also including lower and upper case letters.</i>

[1.5 points] 2. Several users of PayDude.com have been hacked and have each lost \$10 from their balances. They noticed a suspicious transaction to a user named *Law* with the user ID *1234* after visiting piracy websites such as The Pirate Bait. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit and how often it succeeds.

The exploit is a CSRF attack and the visited page contains a form that is auto submitted as soon as the page is loaded by the browser:

The form is the following:

```
<form method="POST" action="/send_money">
  <input type="hidden" name="amount" value="10$">
  <input type="hidden" name="receiver_id" value="1234">
  <input type="hidden" name="CSRF" value="11">
  <input type="submit" value="Get money" onload=click()>
</form>
```

In this form, the CSRF token is hardcoded to 11 and whenever the victim will submit this form, the attacker will have 1% (1/100) of success. Considering the large amount of users, the fact that the exploit can be insert into several pages and with different values for the CSRF token, this attack scenario is highly realistic.

N.B.: the solution doesn't require the form to be specified nor a so detailed explanation on the success of the attack. A less detailed solution can be awarded with max points.

[1.5 points] 3. Can you leak the hashed password of the user *Law* with user_id *1234*? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

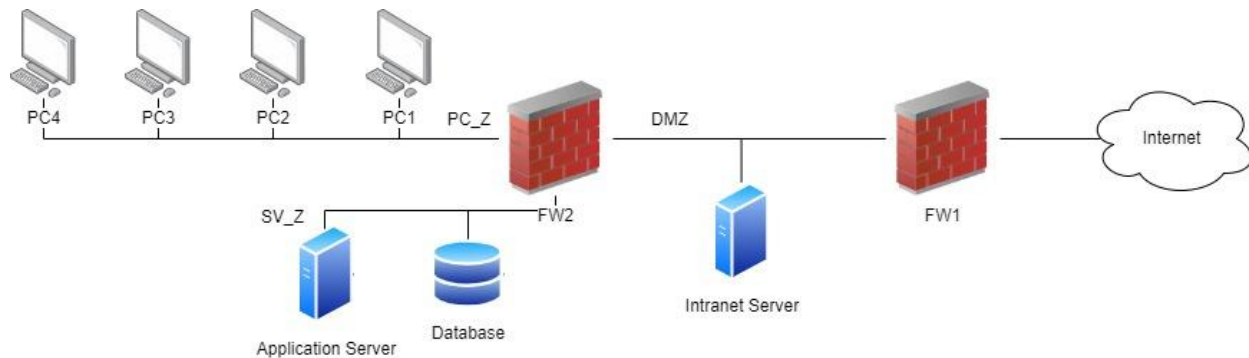
It is possible to leak the hashed password through the SQLI at line 17. We put a low amount so that the condition returns False and we perform an UNION to retrieve the password. The query is the following:

Query = " 0 UNION SELECT hashed_password FROM USERS WHERE user_id = 1234"

Exercise 5 - Network Security (6 points)

ZenTech Corporation is a small company, whose local network layout is the following:

ZT's employees can work from their PCs in PC_Z, which require full communication with the internet and with the intranet server. The intranet server requires access to data through the application server on port 88888, which requires to be connected to the DB through port 55555.



[1 point] 1. Write the firewalls rules, assuming the firewalls to be stateful packet filter:

Firewall	Src IP (or entity names)	Src PORT	Direction of the 1st packet	Dst IP (or entity names)	Dst PORT	Policy	Description
FW1 (example)	10.0.0.1 (example)	ANY	zone 1 -> zone 2	192.168.0.2 (example)	443	ALLOW	(example: the X server in zone 1 cannot contact the Y server)
FW1/2	ALL	ANY	ALL	ALL	ANY	DENY	Default deny
FW2	PCs IPs	ALL	PC_Z->DMZ	ALL	ALL	ALLOW	Full internet access for PCs
FW1	PCs IPs	ALL	DMZ->Internet	ALL	ALL	ALLOW	Full internet access for PCs
FW2	Intranet Sv IP	88888	DMZ->SV_Z	Application Server	88888	ALLOW	Intranet sv to application sv access

[2 points] 2. The company would like to enable the employees to work from home. However, they want to ensure that the employees laptops, provided to them by the company, and on which they are only allowed to do work-related stuff, have the same secure connection that they would have in PC_Z. Note that the company is not worried about bandwidth usage. What should the company implement? Would this require new network equipment or new machines? Would it require adapting the firewall rules? Justify your configuration choices.

The company could implement a Full Tunnel VPN. In fact, a full tunnel VPN ensures all traffic from the laptop passes through it, and since the employees can only do work stuff on the laptops and the company is not worried about bandwidth, which is one of the negative sides of full tunneling, this should be a better choice than split-tunnel VPN.

The Company should implement a VPN server, located on the PC_Z subnetwork, so that once the employees pass through the VPN, the firewalls ensure the same level of security.

The firewall rules should be only slightly adapted, we'd need a FW1 and FW2 rule that allows external communication to reach the VPN server IP, on the custom port that the VPN protocol uses.

[2 points] 3. Lately, employees at ZenTech Corporation have reported sluggish network performance and stolen credentials. The IT department, suspecting network anomalies, has extracted some LAN logs. As a network security analyst, you have been assigned the task of investigating these logs. Your objective is to understand the interactions between the devices and identify any potential security threats or network misconfigurations. Here are the list of IPs and MACs associated with the relevant machines and an extract of the log.

PC1 - 192.168.1.4 - 00:0A:BC:45:78:90

PC2 - 192.168.1.5 - 00:0A:BC:12:34:56

PC4 - 192.168.1.7 - 00:0A:BC:23:45:67

Gateway (FW2) - 192.168.1.1 - 00:0A:BC:78:90:01

	Source IP	Source MAC	Dest. IP	Dest. MAC	Protc	Info
	192.168.1.4	00:0A:BC:45:78:90	192.168.1.1	00:0A:BC:78:90:01	ICMP	"Ping Request"
	192.168.1.1	00:0A:BC:78:90:01	192.168.1.4	00:0A:BC:45:78:90	ICMP	"Ping Reply"
	192.168.1.1	00:0A:BC:78:90:01	255.255.255.255	FF:FF:FF:FF:FF:FF	ARP	"Who has 192.168.1.5? Tell 192.168.1.1"
	192.168.1.7	00:0A:BC:23:45:67	192.168.1.1	00:0A:BC:78:90:01	ARP	"192.168.1.5 is at 00:0A:BC:23:45:67"
	192.168.1.5	00:0A:BC:12:34:56	192.168.1.1	00:0A:BC:78:90:01	ARP	"192.168.1.5 is at 00:0A:BC:12:34:56"
	192.168.1.4	00:0A:BC:45:78:90	8.8.8.8	--:--:--:--:--:--	DNS	"www.acmecorp.com"
	192.168.1.5	00:0A:BC:12:34:56	255.255.255.255	FF:FF:FF:FF:FF:FF	ARP	"Who has 192.168.1.1? Tell 192.168.1.5"
	192.168.1.7	00:0A:BC:23:45:67	192.168.1.5	00:0A:BC:12:34:56	ARP	"192.168.1.1 is at 00:0A:BC:23:45:67"
	192.168.1.1	00:0A:BC:78:90:01	192.168.1.5	00:0A:BC:12:34:56	ARP	"192.168.1.1 is at 00:0A:BC:78:90:01"
	192.168.1.4	00:0A:BC:45:78:90	192.168.1.1	00:0A:BC:78:90:01	HTTP	"GET /intranet/news"
	192.168.1.5	00:0A:BC:12:34:56	192.168.1.1	00:0A:BC:23:45:67	HTTP	"GET /intranet/dashboard"
	192.168.1.7	00:0A:BC:23:45:67	192.168.1.1	00:0A:BC:78:90:01	HTTP	"GET /intranet/dashboard"
	192.168.1.1	00:0A:BC:78:90:01	192.168.1.7	00:0A:BC:23:45:67	HTTP	"200 OK (/intranet/dashboard)"
	192.168.1.7	00:0A:BC:23:45:67	192.168.1.5	00:0A:BC:12:34:56	HTTP	"200 OK (/intranet/dashboard)"

What is the name of the network attack implemented? Explain briefly how it works and why it is possible. What is the goal of the attacker?

The attack is ARP Poisoning / Spoofing. The attacker answers to the gateway as if he was PC2, and to PC2 as if he was the gateway. It is possible because ARP is not authenticated and the attacker is answering faster than the victim, being in the same network.
The goal of the attacker is that of becoming a Man in the middle, as visible from the end of the log where he forwards information between PC2 and the Gateway.

[1 point] 4. How could the company mitigate the attacks?

Use HTTPS instead of HTTP, so that even if the attacker manages to act as a MITM he cannot read or modify the context of the packets.

Exercise 6 - Malware Security (6 points)

Nautilus Institute was recently targeted by a new malware from mhackeroni Inc. We obtained and analyzed two samples of the malicious software identifying some intriguing behaviors. The most notable sections are shown in the following assembly codes.

Sample 1

```
112a:  push  rbp
112b:  mov   rbp, rsp
112e:  mov   DWORD PTR [rbp-0x8], 0x2
1135:  mov   DWORD PTR [rbp-0x4], 0x3
113c:  mov   DWORD PTR [rbp-0x10], 0x1
1143:  mov   DWORD PTR [rbp-0xc], 0x0
114a:  jmp   115a <main+0x30>
114c:  mov   eax, DWORD PTR [rbp-0x10]
114f:  imul  eax, DWORD PTR [rbp-0x8]
1153:  mov   DWORD PTR [rbp-0x10], eax
1156:  add   DWORD PTR [rbp-0xc], 0x1
115a:  mov   eax, DWORD PTR [rbp-0xc]
115d:  cmp   eax, DWORD PTR [rbp-0x4]
1160:  jnl   114c <main+0x22>
1162:  mov   eax, 0x0
1167:  pop   rbp
1168:  ret
```

Sample 2

```
112a:  push  rbp
112b:  mov   rbp, rsp
112e:  sub   rsp, 0x10
1132:  mov   DWORD PTR [rbp-0xc], 0x2
1139:  mov   DWORD PTR [rbp-0x8], 0x3
1140:  mov   edx, DWORD PTR [rbp-0x8]
1143:  mov   eax, DWORD PTR [rbp-0xc]
1146:  mov   esi, edx
1148:  mov   edi, eax
114a:  call  1119 <pow>
114f:  mov   DWORD PTR [rbp-0x4], eax
1152:  mov   eax, 0x0
1157:  leave
1158:  ret
```

1) [2 points] Which technique does this malware employ? Please explain how this is implemented in this particular case. Please note: naming wrong techniques will make you lose points.

The malware employs a metamorphic technique. While Sample 2 makes a call to perform the power operation at address 0x114a, Sample 1 accomplishes the same using a loop cycle, running from address 0x114c to address 0x1160.

2) [2 points] What type of analysis would you apply to **DETECT** the malware? Please provide a detailed motivation of your answer and specify all the assumptions you have made.

It is not possible to apply static detection but we can apply dynamic detection techniques.

See slide 46 (Malicious Software)...

A few weeks later, we identified a new variant of the same malware. We executed the command “strings” on both Sample 1 from points 1 and 2 and the newly identified sample, which we are calling Sample 3. The results of this command can be seen in the table below.

Note: The "strings" command extracts and displays all the strings, or those assumed to be strings, from a given binary.

Sample 1	Sample 3
/lib64/ld-linux-x86-64.so.2 _ITM_deregisterTMCloneTable __gmon_start__ _ITM_registerTMCloneTable __cxa_finalize __libc_start_main libm.so.6 libc.so.6 GLIBC_2.2.5 GLIBC_2.34 GCC: (GNU) 13.2.1 20230801 test.c _DYNAMIC __GNU_EH_FRAME_HDR _GLOBAL_OFFSET_TABLE_ __libc_start_main@GLIBC_2.34 _ITM_deregisterTMCloneTable	IuDSWH s2V^ XAVAWPH YT_j AY^_ D\$ [I UPX!u IT\$} _w5v SG(lm []A\A] USMhL tL G +xHf p(E1 hW 1 I5(Ag

<code>_edata</code> <code>_fini</code> <code>__data_start</code> <code>__gmon_start__</code> <code>main</code>	<code>@bQs</code> <code>`]K/</code> <code>k1(~</code> <code>so5v</code> <code>A(I[u</code>
--	--

3) [2 points] Which technique does the new variant employ? What technique would you apply to **DETECT** the malware? Please provide a detailed motivation of your answer and specify all the assumptions you have made.

The new variant utilizes polymorphism. Specifically, it leverages packing to obscure the malware's content, at least the strings. Static detection is ineffective for this variant; only dynamic analysis can be applied.